

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

ОТЧЕТ

Лабораторная работа №2

Выполнила:

Красноперова Е. Д.

Группа: К3443

Проверил:

Добряков Д. И.

Санкт-Петербург,

2026 г.

1. Задание

Опираясь на архитектуру, сформированную в ДЗ4, реализовать разделение существующего backend-приложения на микросервисы.

2. Исходный монолит

За основу взят код ЛР1 на Express, TypeScript, TypeORM и PostgreSQL. В ЛР2 доменные области разделены по микросервисам, а междоменное взаимодействие выполняется через HTTP API.

3. Архитектура ЛР2

API Gateway — 3000, UserService — 3001, RecipeService — 3002, SocialService — 3003. Gateway является единой точкой входа. Каждый бизнес-сервис использует собственную PostgreSQL database.

4. Database-per-service

UserService владеет users; RecipeService — recipes, steps, ingredients, recipe_ingredients; SocialService — likes, comments, saved_recipes, subscriptions. Межсервисные user_id и recipe_id не являются ORM-связями с чужими БД; существование объектов проверяется через внутренние HTTP endpoints.

5. UserService

Реализованы регистрация, вход, JWT, получение и изменение профиля, публичный профиль и защищённые внутренние endpoints /internal/users/... с x-service-token.

6. RecipeService

Реализованы список, получение, создание, изменение и удаление рецептов, получение рецептов текущего пользователя через /api/recipes/mine, а также внутренние endpoints проверки существования рецепта. Автор хранится как author_id, без ORM-связи с User.

7. SocialService

Реализованы лайки, комментарии, сохранения и подписки. Перед операциями, требующими объектов других доменов, выполняется межсервисная проверка через UserService и RecipeService.

8. Межсервисное взаимодействие

Используются синхронные HTTP REST-запросы и JSON. Для внутренних запросов применяется сервисный токен x-service-token.

9. API Gateway

Gateway работает на порту 3000, не имеет бизнес-БД и маршрутизирует запросы в три микросервиса. Authorization передаётся целевому сервису.

10. Авторизация

Пользовательская авторизация выполняется с JWT. Внутренние сервисные запросы защищены отдельным x-service-token.

11. Базы данных и запуск

Используются recipes_users, recipes_recipes и recipes_social. Сервисы запускаются на 3001–3003, Gateway — на 3000

12. Проверка функционального сценария

Выполнены curl-проверки основных операций и обработки ошибок.

Проверка	Результат
GET /api/users/me без JWT	401 UNAUTHORIZED
GET /api/recipes/99999	404 RECIPE_NOT_FOUND
Пустой комментарий	422 VALIDATION_ERROR
UserService internal: правильный/неправильный token	200 / 401
RecipeService internal: правильный/неправильный token	200 / 401
SocialService internal: правильный/неправильный token	200 / 401
Лайк: добавить → повторить → удалить → добавить	200 → 409 → 200 → 200
Сохранение: повторить → удалить → сохранить	409 → 200 → 200
Комментарий: создать → получить → изменить	201 → 200 → успешно
CRUD рецептов и ownership	основные операции и 403 проверены

Отдельно в финальной серии тестов не выполнялись удаление комментария, попытка изменения чужого комментария и полный набор операций подписок; поэтому эти сценарии не представлены как отдельно подтверждённые тестами.

13. Вывод

В LP2 монолитное backend-приложение разделено на UserService, RecipeService и SocialService перед API Gateway. Для сервисов выделены отдельные PostgreSQL database. Межсервисное взаимодействие реализовано через REST/JSON и защищённые внутренние endpoints, пользовательская авторизация — через JWT. Выполненные проверки подтвердили корректную маршрутизацию, основные CRUD- и социальные операции, проверки владельца и обработку ошибок 401, 404 и 422.